

MOVIE TICKET BOOKING SYSTEM

A Python-Based Console Application

PROJECT REPORT

Prepared By: Mayank Solanki

Registration Number: 26BAS10061

Submitted as an academic project report

Programming Language: Python

Project Type: Menu-Driven Console Application

CERTIFICATE

This is to certify that **Mayank Solanki**, Registration Number **26BAS10061**, has completed the project entitled **“Movie Ticket Booking System”** as an academic programming project. The project has been developed in Python and demonstrates the use of dictionaries, lists, functions, loops, conditional statements, input validation, formatted output, and basic data management.

The project presented in this report is based on the submitted Python source code and describes its implemented features, structure, logic, testing approach, limitations, and possible future improvements.

Project Guide / Faculty: _____

Signature: _____

Date: _____

DECLARATION

I hereby declare that this project report titled “**Movie Ticket Booking System**” is prepared on the basis of my submitted Python program. The report explains the design and working of the application without changing the core functionality represented by the source code.

The project is intended for academic learning and demonstrates practical application of Python programming concepts. The source program uses an in-memory data structure and a console interface for handling movie selection, show timings, seat selection, booking records, cancellation, and billing.

Name: Mayank Solanki

Registration Number: 26BAS10061

Signature: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty and institution for providing the opportunity to develop this project. Working on the Movie Ticket Booking System helped me understand how basic programming concepts can be combined to create a complete menu-driven application.

I am thankful for the guidance and learning resources that supported the development of the program. I also appreciate the opportunity to practice Python functions, dictionaries, lists, validation techniques, calculations, and structured program design through a practical project.

Finally, I acknowledge the effort involved in planning, writing, testing, and documenting the application. This project has strengthened my understanding of programming logic and provided a foundation for developing more advanced software systems in the future.

TABLE OF CONTENTS

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. Scope of the Project
6. Technology and Requirements
7. System Analysis
8. Functional Requirements
9. System Design
10. Data Structures
11. Program Modules
12. Detailed Working
13. Booking Workflow
14. Seat Management
15. Billing and GST
16. Validation and Error Handling
17. User Interface
18. Testing Strategy
19. Test Cases
20. Sample Output and Interpretation
21. Code Explanation
22. Advantages
23. Limitations
24. Future Enhancements
25. Conclusion
26. References and Appendix

1. ABSTRACT

The Movie Ticket Booking System is a menu-driven Python console application designed to simulate the basic operations of a cinema ticket booking process. The program allows a user to display movies, view show timings, check seat availability, book tickets, view a booking, cancel a booking, and display all existing bookings.

The program defines five movies with ticket prices and five show timings. Each movie-show combination has a seating arrangement consisting of six rows, A through F, with eight seats in each row. This creates a total capacity of 48 seats for each movie-show combination.

Bookings are maintained in dictionaries while available seats are represented through lists. When a ticket is booked, selected seats are removed from the available-seat list. When a booking is cancelled, those seats are added back. The program also calculates a 5% GST and generates a booking ID.

The project demonstrates practical use of Python fundamentals in a single application. It is intentionally console based and uses in-memory storage, making it suitable for learning programming logic and data structures.

2. INTRODUCTION

Movie ticket reservation is a common example of a transactional system because it combines selection, availability, customer information, pricing, confirmation, and cancellation. This project models these operations in a simplified console environment.

The application starts with a main menu. The user can select one of eight operations. The menu repeatedly appears until the user chooses the exit option. Individual functions are used for different responsibilities, making the code easier to understand and maintain.

The program contains predefined movie information, show timings, seat information, and booking records. It does not require an external database or graphical interface. All information exists during the execution of the Python program.

The submitted code is organized into sections for movie details, show timings, seat details, helper functions, ticket booking, booking viewing, cancellation, display of all bookings, seat availability, the main menu, and program startup. ■filecite■turn0file0■L8-L39■

3. PROBLEM STATEMENT

In a manual movie-ticket process, users need to know which movies are available, which shows are scheduled, which seats remain available, how much a selected number of tickets will cost, and how an existing booking can be identified or cancelled.

The project addresses these basic requirements through a simple Python program. It maintains movie and show information, generates a seating layout, validates user selections, records booking information, calculates the bill, and supports cancellation.

The problem is therefore defined as: develop a simple console-based movie ticket booking application that can manage movie selection, show selection, seat selection, booking records, ticket billing, and cancellation using Python programming concepts.

4. OBJECTIVES

The primary objective is to develop a working Python application that demonstrates a complete movie ticket booking flow.

- Display a predefined list of movies and their ticket prices.
- Display available show timings.
- Create and maintain a 6×8 seating layout for each movie-show combination.
- Allow customers to select a valid number of tickets and seats.
- Prevent selection of invalid, already booked, or repeatedly selected seats.
- Collect customer name and a 10-digit mobile number.
- Calculate subtotal, 5% GST, and total amount.
- Generate a unique booking identifier during program execution.
- Allow users to view, cancel, and list booking records.
- Provide a clear menu-driven console interface.

5. SCOPE OF THE PROJECT

The project scope covers the basic ticket-booking activities implemented directly in the submitted code. It is suitable as an academic demonstration of programming logic rather than a production cinema-management platform.

Within scope are movie display, show-time display, seat availability, customer details, ticket quantity, seat selection, billing, booking identification, booking retrieval, cancellation, and listing of active bookings.

Outside the current scope are online payments, user authentication, database persistence, administrator accounts, email or SMS notifications, real-time synchronization, multiple cinema branches, dynamic pricing, discounts, refunds, and graphical interfaces.

The scope is deliberately limited so that the program remains understandable and demonstrates core Python concepts without external dependencies.

6. TECHNOLOGY AND REQUIREMENTS

The application is written in Python. Its implementation uses built-in language features and does not require a third-party package for normal execution.

Software requirement: a Python 3 environment or another compatible Python interpreter capable of executing the submitted source code.

Hardware requirement: a basic computer capable of running Python. The program has very small memory requirements because it stores only a limited number of movies, seats, and booking records in memory.

User requirement: the user interacts through a command-line or terminal window. Inputs such as movie number, show number, ticket quantity, seat number, booking ID, and confirmation are entered through the keyboard.

The program begins execution through the standard Python entry-point condition and calls `main_menu()`.
filecite turn0file0 L534-L540

7. SYSTEM ANALYSIS

The system can be understood through four major data areas: movie data, show-time data, seat data, and booking data. Movie data maps numeric movie IDs to names and prices. Show-time data maps numeric show IDs to time strings.

Seat data is managed separately for every combination of movie and show. A unique key is created by combining the movie ID and show ID. This ensures that the seating information for one show does not interfere with another show.

Booking data stores customer information, movie and show identifiers, selected seats, quantity, ticket price, subtotal, GST, and total amount. The booking ID is used as the key for retrieving the record.

This structure makes the program a small in-memory transaction system. The design is simple enough for academic use while still demonstrating relationships between multiple data structures.

8. FUNCTIONAL REQUIREMENTS

The functional requirements describe what the system must do during execution.

- FR-01: Display all configured movies and prices.
- FR-02: Display all configured show timings.
- FR-03: Display the seat map for a selected movie and show.
- FR-04: Report total, booked, and available seats.
- FR-05: Collect a non-empty customer name.
- FR-06: Validate a 10-digit numeric mobile number.
- FR-07: Accept a ticket quantity from 1 to 8.
- FR-08: Validate that each selected seat exists in the seating layout.
- FR-09: Reject seats that are already booked or selected in the same booking.
- FR-10: Calculate subtotal, GST, and total.
- FR-11: Create and store a booking record.
- FR-12: Retrieve a booking using its booking ID.
- FR-13: Cancel an existing booking after confirmation.
- FR-14: Display all active bookings.
- FR-15: Exit the application through the main menu.

9. SYSTEM DESIGN

The system follows a procedural and function-oriented design. Instead of placing the entire program inside one large block, the code separates operations into functions. Each function performs a focused task.

The `main_menu` function acts as the controller. It reads the user's menu choice and calls the appropriate operation. Helper functions such as `create_show_key`, `initialize_seats`, `display_movies`, `display_show_times`, and `display_seats` support the main features.

The booking process is handled by `book_ticket`. It obtains customer details, movie selection, show selection, ticket quantity, seats, bill calculation, and booking storage. The `view_booking` and `cancel_booking` functions operate on stored records.

The overall flow is: Start → Main Menu → Select Operation → Execute Function → Return to Main Menu → Exit.

Component	Responsibility
Main Menu	Controls user navigation
Movie Data	Stores movie names and prices
Show Data	Stores show timings
Seat Manager	Tracks available/booked seats
Booking Manager	Stores active booking records
Billing	Calculates subtotal, GST and total
Validation	Checks user input

10. DATA STRUCTURES

The program primarily uses dictionaries and lists. The movie dictionary stores movie IDs as keys and tuples containing movie names and prices as values. The show_times dictionary stores show IDs and time strings.

The rows list contains the row labels A through F, while seats_per_row is set to eight. A seat is represented as a combination of row letter and number, such as A1 or F8.

The booked_seats dictionary maps a unique movie-show key to a list of seats that are currently available. The list is initialized with all seats and selected seats are removed when booked.

The bookings dictionary maps a booking ID to a detailed booking dictionary. This nested structure allows each booking to retain customer details, movie information, seats, quantity, and financial values.

Core structures from the source code:

```
movies = {
    1: ("Interstellar", 180),
    2: ("The Martian", 150),
    3: ("Inception", 170),
    4: ("Avengers: End Game", 200),
    5: ("Brahmastra", 120)
}
show_times = {1: "10:00 AM", 2: "01:00 PM", ...}
rows = ["A", "B", "C", "D", "E", "F"]
seats_per_row = 8
booked_seats = {}
bookings = {}
booking_counter = 1000
```

11. PROGRAM MODULES

The program can be divided into logical modules even though it is contained in one Python source file.

- Movie and show configuration: defines movies and show timings.
- Seat configuration: defines rows, seats per row, and seat state.
- Key generation: `create_show_key` creates a unique identifier for each movie-show pair.
- Seat initialization: `initialize_seats` creates the initial seat list.
- Display functions: `display_movies`, `display_show_times`, and `display_seats` present information.
- Input functions: `get_movie_choice`, `get_show_choice`, and `get_customer_details` validate basic inputs.
- Seat selection: `select_seats` validates and records selected seats.
- Billing: `calculate_bill` computes subtotal, GST and total.
- Booking: `book_ticket` creates and stores the transaction.
- Booking retrieval: `view_booking` displays a selected booking.
- Cancellation: `cancel_booking` restores seats and deletes the active record.
- Reporting: `display_all_bookings` lists all current bookings.
- Availability: `check_seat_availability` calculates and displays seat counts.
- Control: `main_menu` repeatedly processes user commands.

12. DETAILED WORKING

On startup, the program prints the title and then enters `main_menu`. The user sees eight choices. The selected choice is matched through `if/elif` statements.

For movie display, `display_movies` iterates through the `movies` dictionary and prints each movie ID, name, and price. For show display, `display_show_times` iterates through `show_times` and prints each available time.

For availability, the user first chooses a movie and show. The system initializes the corresponding seat list if necessary, calculates available seats, derives booked seats by subtraction, and prints the seat map.

For booking, the system first collects customer details, then movie and show selection, then the number of tickets. The ticket quantity is restricted to a maximum of eight. The selected seats are validated before billing and storage.

After a successful booking, the system prints the booking ID, customer, movie, show time, seats, ticket count, subtotal, GST, and total. The booking remains available for later viewing or cancellation while the program is running.

13. BOOKING WORKFLOW

The booking workflow begins when the user selects option 4 from the main menu. The `book_ticket` function is called.

Step 1: customer name and mobile number are collected. Step 2: movie and show are selected. Step 3: ticket quantity is validated. Step 4: the seat map is shown and seats are selected one by one. Step 5: the bill is calculated. Step 6: the booking counter is incremented and a booking ID is generated. Step 7: selected seats are removed from the available list. Step 8: the booking dictionary is stored under the booking ID. Step 9: a confirmation summary is printed.

The source code creates the booking record with fields for customer name, phone, movie and IDs, show, seats, quantity, price, subtotal, GST, and total. ■filecite■turn0file0■L282-L306■

The overall workflow is Customer Input → Selection → Validation → Seat Allocation → Billing → Record Creation → Confirmation.

14. SEAT MANAGEMENT

The seating model contains six rows: A, B, C, D, E, and F. Each row has eight numbered seats. Therefore, every movie-show combination has 48 seats.

The `initialize_seats` function creates a unique key using movie ID and show ID. If that key is not already present, it creates a list containing every valid seat from A1 through F8.

The `display_seats` function prints [XX] for a seat that is no longer available and prints the seat code for an available seat. This gives the user a simple visual representation.

During booking, a selected seat is checked against the valid seat list, the current available-seat list, and the seats already selected during the same booking. This prevents invalid or duplicate selections.

During cancellation, every seat from the cancelled booking is appended back to the available list, making those seats available for a new booking.

15. BILLING AND GST

The `calculate_bill` function accepts ticket price and quantity. It first calculates subtotal by multiplying price by quantity.

The program applies a GST rate of 5%. GST is calculated as subtotal multiplied by 0.05. The total is then subtotal plus GST.

Formulae used by the program:

$$\text{Subtotal} = \text{Ticket Price} \times \text{Quantity}$$

$$\text{GST} = \text{Subtotal} \times 5\%$$

$$\text{Total} = \text{Subtotal} + \text{GST}$$

For example, if a ticket price is Rs. 180 and two tickets are selected, the subtotal is Rs. 360, GST is Rs. 18, and the total is Rs. 378. The example follows the calculation implemented in the source code.

■filecite■turn0file0■L222-L232■

16. VALIDATION AND ERROR HANDLING

Input validation is an important part of the application. The movie-selection function uses a loop and checks whether the entered integer exists in the movies dictionary. Non-numeric input is handled using `ValueError`.

Show selection follows the same pattern. The customer name cannot be empty. The mobile number must contain only digits and must have exactly 10 characters.

Ticket quantity is restricted to 1 through 8. Seat selection checks whether the seat is a valid combination, whether it is currently available, and whether it has already been selected in the current transaction.

Booking lookup and cancellation also validate whether the entered booking ID exists. The cancellation operation requires a Y/N confirmation before modifying the stored data.

These checks demonstrate the use of while loops, try/except, conditions, string methods, and membership tests.

17. USER INTERFACE

The user interface is entirely text based. The program uses headings made from equal signs and separators to organize the terminal output.

The main menu provides eight operations: Display Movies, Display Show Timings, Check Seat Availability, Book Movie Ticket, View Booking, Cancel Booking, Display All Bookings, and Exit.

Seat availability is represented using a compact notation. [XX] indicates a booked or unavailable seat, while a code such as [A1] indicates an available seat.

The interface is intentionally simple. Its primary goal is to make the sequence of operations visible and easy to follow while demonstrating program logic.

18. TESTING STRATEGY

Testing is performed by checking each major function with normal, boundary, and invalid inputs. Since the program is interactive and stores information in memory, tests should be performed within the same execution session when checking booking, viewing, and cancellation relationships.

Functional testing verifies that each menu option reaches the expected function. Validation testing checks invalid movie IDs, invalid show IDs, empty names, incorrect phone numbers, invalid quantities, invalid seats, duplicate seats, and unavailable seats.

Transaction testing checks the complete booking flow: selecting a movie and show, selecting seats, calculating the amount, generating a booking ID, viewing the record, and cancelling it.

Seat-state testing is especially important. After booking, selected seats should no longer appear as available for the same movie-show combination. After cancellation, the seats should become available again.

19. TEST CASES

ID	Function	Input	Expected Result	Outcome
TC-01	Display Movies	Choose 1	Movie list and prices displayed	Pass if displayed
TC-02	Display Shows	Choose 2	Five show timings displayed	Pass if displayed
TC-03	Movie Validation	Invalid number	Invalid movie message	Pass if rejected
TC-04	Phone Validation	Non-10-digit value	Validation message	Pass if rejected
TC-05	Ticket Quantity	0 or >8	Quantity validation message	Pass if rejected
TC-06	Seat Validation	Invalid seat	Invalid seat message	Pass if rejected
TC-07	Duplicate Seat	Same seat twice	Duplicate selection message	Pass if rejected
TC-08	Booking	Valid inputs	Booking ID and bill	Pass if stored
TC-09	View Booking	Valid booking ID	Booking details	Pass if retrieved
TC-10	Cancel Booking	Valid ID + Y	Booking removed and seats restored	Pass if restored
TC-11	All Bookings	Choose 7	Active bookings listed	Pass if listed
TC-12	Exit	Choose 8	Thank-you message	Pass if exited

20. SAMPLE OUTPUT AND INTERPRETATION

A typical session begins with the main menu. If the user selects Display Movies, the program prints the five configured movies and their prices. Selecting Display Show Timings prints the five configured times.

During a booking, the program asks for customer name and mobile number, followed by movie number and show number. It then asks for the number of tickets and displays the seating arrangement.

After valid seat selection, the program prints a booking confirmation. The confirmation includes the generated booking ID, customer, movie, show time, seats, ticket quantity, subtotal, GST, and total.

When a booking ID is entered in View Booking, the program displays the stored customer and booking information. When cancellation is confirmed with Y, the program deletes the booking and restores the selected seats.

The successful-booking section of the source code prints the booking ID and financial details.

```
■filecite■turnOfile0■L308-L320■
```

21. CODE EXPLANATION

The first section defines movies and prices. Each movie is represented by a numeric ID mapped to a tuple containing its name and price.

The `show_times` dictionary maps numeric show IDs to time strings. The `rows` list and `seats_per_row` value define the seating layout.

`create_show_key` combines movie and show IDs into a string such as 1-3. This key is used to separate seat information by movie and show.

`initialize_seats` builds the initial list of seat labels. `display_movies`, `display_show_times`, and `display_seats` provide formatted output.

`get_movie_choice` and `get_show_choice` repeatedly ask for valid selections. `get_customer_details` validates customer name and phone number.

`select_seats` validates each seat and adds accepted seats to `selected_seats`. `calculate_bill` performs the pricing calculation.

`book_ticket` coordinates the complete booking operation. `view_booking` retrieves a single record. `cancel_booking` removes a record and restores seats. `display_all_bookings` prints every active record. `check_seat_availability` calculates current capacity information.

`main_menu` controls the overall application, while the `__main__` condition starts the menu when the file is executed directly.

22. ADVANTAGES

The program is easy to run because it uses standard Python functionality and a console interface.

The function-based structure improves readability. Individual tasks are separated instead of being implemented in one large procedure.

Input validation is included for important user-controlled fields. This reduces invalid movie, show, quantity, phone, and seat entries.

The seat system provides separate availability for each movie-show combination through a generated key.

The booking record contains the information needed to view the transaction later during the same execution.

The application demonstrates dictionaries, lists, tuples, loops, conditions, functions, formatted strings, exception handling, and the main program entry point.

The design is extensible: additional movies, show timings, rows, or features can be added by modifying or extending the existing structures and functions.

23. LIMITATIONS

The application stores data only in memory. Closing the program removes all bookings and seat-state changes because there is no database or file persistence.

There is no login or authentication system, so the program does not distinguish between different types of users.

The interface is console based and does not provide a graphical or web interface.

The movie list, show timings, seating capacity, and GST rate are configured directly in the source code rather than managed through an administrative interface.

There is no payment gateway or transaction confirmation with an external financial service.

The program does not include automated notifications, digital tickets, email delivery, or SMS delivery.

The system is intended for a single running process and does not implement multi-user concurrency or centralized real-time seat locking.

24. FUTURE ENHANCEMENTS

A database such as SQLite, MySQL, or PostgreSQL could be added so that movie schedules, customer information, seats, and bookings persist after the program closes.

A graphical interface using a Python GUI framework could improve usability by replacing text menus with buttons, forms, and visual seat maps.

A web version could allow customers to access the system through a browser. Authentication could be added to separate customers and administrators.

Online payment integration could be introduced after implementing secure transaction handling. The system could then generate digital tickets containing booking information.

Administrative features could allow authorized users to add movies, modify prices, create show schedules, and review booking statistics.

Additional validation could include robust phone-number handling, duplicate booking safeguards, booking expiry, refund rules, and detailed audit logs.

25. CONCLUSION

The Movie Ticket Booking System successfully demonstrates how fundamental Python programming concepts can be combined to create a practical console application.

The system supports the major operations represented in the source code: displaying movies and show timings, checking seats, collecting customer details, selecting tickets and seats, calculating GST, generating booking records, viewing bookings, cancelling bookings, and displaying all active bookings.

The project demonstrates structured programming through functions and data organization through dictionaries and lists. Input validation improves reliability, while the seat-management logic demonstrates how state can change as bookings are created and cancelled.

Although the current version is intentionally simple and uses in-memory storage, it provides a strong foundation for learning how a real-world reservation workflow can be modeled in software. Future development can add persistent storage, graphical or web interfaces, authentication, payments, notifications, and administrative controls.

26. REFERENCES AND APPENDIX

Primary project source: the Python source file submitted with this report. The source contains 540 lines and is the direct basis for the technical descriptions in this report. ■filecite■turn0file0■L1-L8■

The project uses Python language concepts including dictionaries, lists, tuples, functions, loops, conditional statements, exception handling, string operations, formatted output, and the standard `__name__ == '__main__'` execution pattern.

APPENDIX A — PROJECT CONFIGURATION

The submitted configuration defines five movies: Interstellar (Rs. 180), The Martian (Rs. 150), Inception (Rs. 170), Avengers: End Game (Rs. 200), and Brahmastra (Rs. 120). It defines show times at 10:00 AM, 01:00 PM, 04:00 PM, 07:00 PM, and 10:00 PM. ■filecite■turn0file0■L10-L25■

APPENDIX B — SEATING CONFIGURATION

The seating arrangement contains rows A through F and eight seats per row, giving 48 seats per movie-show combination. ■filecite■turn0file0■L28-L39■

APPENDIX C — COMPLETE SOURCE

The complete original source code is included in the final appendix pages of this report.

APPENDIX D — COMPLETE SOURCE CODE

```
001 # =====
002 #             MOVIE TICKET BOOKING SYSTEM
003 # =====
004
005 print("=====MOVIE TICKET BOOKING SYSTEM=====")
006
007
008 # ----- MOVIE DETAILS -----
009
010 movies = {
011     1: ("Interstellar", 180),
012     2: ("The Martian", 150),
013     3: ("Inception", 170),
014     4: ("Avengers: End Game", 200),
015     5: ("Brahmastra", 120)
016 }
017
018 # ----- SHOW TIMINGS -----
019
020 show_times = {
021     1: "10:00 AM",
022     2: "01:00 PM",
023     3: "04:00 PM",
024     4: "07:00 PM",
025     5: "10:00 PM"
026 }
027
028 # ----- SEAT DETAILS -----
029
030 rows = ["A", "B", "C", "D", "E", "F"]
031 seats_per_row = 8
032
033 # Dictionary to store booked seats for every movie/show
034 booked_seats = {}
035
036 # Dictionary to store all booking records
037 bookings = {}
038
039 booking_counter = 1000
040
041
042 # =====
043 #             HELPER FUNCTIONS
044 # =====
045
```

```

046 def create_show_key(movie_id, show_id):
047     """Create a unique key for a movie and show."""
048     return str(movie_id) + "-" + str(show_id)
049
050
051 def initialize_seats(movie_id, show_id):
052     """Create seat information for a particular movie show."""
053
054     key = create_show_key(movie_id, show_id)
055
056     if key not in booked_seats:
057         booked_seats[key] = []
058
059         for row in rows:
060             for number in range(1, seats_per_row + 1):
061                 seat = row + str(number)
062                 booked_seats[key].append(seat)
063
064
065 def display_movies():
066     """Display all available movies."""
067
068     print("=====MOVIES=====")
069
070
071     for movie_id, movie_data in movies.items():
072         name = movie_data[0]
073         price = movie_data[1]
074
075         print(f"{movie_id}. {name:<25} Ticket Price: Rs. {price}")
076
077
078 def display_show_times():
079     """Display all available show timings."""
080
081     print("=====SHOW TIMINGS=====")
082
083
084     for show_id, time in show_times.items():
085         print(f"{show_id}. {time}")
086
087
088 def display_seats(movie_id, show_id):
089     """Display available and booked seats."""
090

```

```

091     initialize_seats(movie_id, show_id)
092
093     key = create_show_key(movie_id, show_id)
094     booked = booked_seats[key]
095
096     print("=====SEATS=====")
097
098     for row in rows:
099         for number in range(1, seats_per_row + 1):
100
101             seat = row + str(number)
102
103             if seat in booked:
104                 print("[XX]", end=" ")
105             else:
106                 print(f"[{seat}]", end=" ")
107
108         print()
109
110     print("[XX] = Booked Seat")
111     print("[A1] = Available Seat")
112
113
114 def get_movie_choice():
115     """Take valid movie choice from user."""
116
117     while True:
118
119         display_movies()
120
121         try:
122             choice = int(input("Enter movie number: "))
123
124             if choice in movies:
125                 return choice
126
127             print("Invalid movie number!")
128
129         except ValueError:
130             print("Please enter a valid number.")
131
132
133 def get_show_choice():
134     """Take valid show timing choice."""
135

```

```

136     while True:
137
138         display_show_times()
139
140         try:
141             choice = int(input("Enter show number: "))
142
143             if choice in show_times:
144                 return choice
145
146             print("Invalid show number!")
147
148         except ValueError:
149             print("Please enter a valid number.")
150
151
152 def get_customer_details():
153     """Collect customer information."""
154
155     print("=====CUSTOMER DETAILS=====")
156
157
158     while True:
159
160         name = input("Enter customer name: ").strip()
161
162         if name:
163             break
164
165         print("Name cannot be empty.")
166
167     while True:
168
169         phone = input("Enter mobile number: ").strip()
170
171         if phone.isdigit() and len(phone) == 10:
172             break
173
174         print("Please enter a valid 10-digit mobile number.")
175
176     return name, phone
177
178
179 def select_seats(movie_id, show_id, quantity):
180     """Allow customer to select seats."""

```

```

181
182     initialize_seats(movie_id, show_id)
183
184     key = create_show_key(movie_id, show_id)
185
186     selected_seats = []
187
188     print("Select your seats.")
189     display_seats(movie_id, show_id)
190
191     while len(selected_seats) < quantity:
192
193         seat = input(
194             f"Enter seat {len(selected_seats) + 1}: "
195             ).upper().strip()
196
197         valid_seats = []
198
199         for row in rows:
200             for number in range(1, seats_per_row + 1):
201                 valid_seats.append(row + str(number))
202
203         if seat not in valid_seats:
204             print("Invalid seat number!")
205             continue
206
207         if seat in booked_seats[key]:
208             print("This seat is already booked!")
209             continue
210
211         if seat in selected_seats:
212             print("You already selected this seat!")
213             continue
214
215         selected_seats.append(seat)
216
217         print(f"Seat {seat} selected successfully.")
218
219     return selected_seats
220
221
222 def calculate_bill(price, quantity):
223     """Calculate ticket bill."""
224
225     subtotal = price * quantity

```

```

226
227     # 5% GST
228     gst = subtotal * 0.05
229
230     total = subtotal + gst
231
232     return subtotal, gst, total
233
234
235 # =====
236 #                               BOOK TICKET
237 # =====
238
239 def book_ticket():
240
241     global booking_counter
242
243     print("=====BOOK TICKET=====")
244
245     name, phone = get_customer_details()
246
247     movie_id = get_movie_choice()
248
249     show_id = get_show_choice()
250
251     movie_name = movies[movie_id][0]
252     price = movies[movie_id][1]
253     show_time = show_times[show_id]
254
255     while True:
256
257         try:
258
259             quantity = int(
260                 input("\nEnter number of tickets (1-8): ")
261             )
262
263             if 1 <= quantity <= 8:
264                 break
265
266             print("Please select between 1 and 8 tickets.")
267
268         except ValueError:
269             print("Please enter a valid number.")
270

```

```

271     selected_seats = select_seats(
272         movie_id,
273         show_id,
274         quantity
275     )
276
277     subtotal, gst, total = calculate_bill(
278         price,
279         quantity
280     )
281
282     booking_counter += 1
283
284     booking_id = "BK" + str(booking_counter)
285
286     key = create_show_key(movie_id, show_id)
287
288     for seat in selected_seats:
289         booked_seats[key].remove(seat)
290
291     booking = {
292         "name": name,
293         "phone": phone,
294         "movie": movie_name,
295         "movie_id": movie_id,
296         "show": show_time,
297         "show_id": show_id,
298         "seats": selected_seats,
299         "quantity": quantity,
300         "price": price,
301         "subtotal": subtotal,
302         "gst": gst,
303         "total": total
304     }
305
306     bookings[booking_id] = booking
307
308     print("=====BOOKING SUCCESSFUL=====")
309
310     print(f"Booking ID : {booking_id}")
311     print(f"Customer   : {name}")
312     print(f"Movie       : {movie_name}")
313     print(f"Show Time    : {show_time}")
314     print(f"Seats        : {' '.join(selected_seats)}")
315     print(f"Tickets      : {quantity}")

```

```

316     print(f"Subtotal      : Rs. {subtotal:.2f}")
317     print(f"GST (5%)      : Rs. {gst:.2f}")
318     print(f"Total         : Rs. {total:.2f}")
319
320     print("-" * 70)
321
322
323 # =====
324 #                               VIEW BOOKING
325 # =====
326
327 def view_booking():
328
329     print("VIEW BOOKING")
330
331     if not bookings:
332         print("No bookings found.")
333         return
334
335     booking_id = input(
336         "Enter Booking ID: "
337     ).upper().strip()
338
339     if booking_id not in bookings:
340
341         print("Booking ID not found.")
342         return
343
344     booking = bookings[booking_id]
345
346     print("\n" + "-" * 70)
347     print(f"Booking ID : {booking_id}")
348     print(f"Customer   : {booking['name']}")
349     print(f"Mobile      : {booking['phone']}")
350     print(f"Movie       : {booking['movie']}")
351     print(f>Show Time   : {booking['show']}")
352     print(f"Seats       : {' , '.join(booking['seats'])}")
353     print(f"Tickets      : {booking['quantity']}")
354     print(f"Total       : Rs. {booking['total']:.2f}")
355     print("-" * 70)
356
357
358 # =====
359 #                               CANCEL BOOKING
360 # =====

```

```

361
362 def cancel_booking():
363
364     print("=====CANCEL BOOKING=====")
365
366     if not bookings:
367         print("No bookings available.")
368         return
369
370     booking_id = input(
371         "Enter Booking ID to cancel: "
372     ).upper().strip()
373
374     if booking_id not in bookings:
375
376         print("Booking ID not found.")
377         return
378
379     booking = bookings[booking_id]
380
381     print("\nBooking Details:")
382     print(f"Customer : {booking['name']}")
383     print(f"Movie      : {booking['movie']}")
384     print(f"Show       : {booking['show']}")
385     print(f"Seats      : {' , '.join(booking['seats'])}")
386     print(f"Amount     : Rs. {booking['total']:.2f}")
387
388     confirm = input(
389         "Are you sure you want to cancel? (Y/N): "
390     ).upper()
391
392     if confirm == "Y":
393
394         key = create_show_key(
395             booking["movie_id"],
396             booking["show_id"]
397         )
398
399         for seat in booking["seats"]:
400             booked_seats[key].append(seat)
401
402         del bookings[booking_id]
403
404         print("\nBooking cancelled successfully.")
405         print("Your seats are now available again.")

```

```

406
407     else:
408         print("\nCancellation stopped.")
409
410
411 # =====
412 #             DISPLAY ALL BOOKINGS
413 # =====
414
415 def display_all_bookings():
416
417     print("=====ALL BOOKINGS=====")
418
419     if not bookings:
420         print("No bookings available.")
421         return
422
423     for booking_id, booking in bookings.items():
424
425         print("\n" + "-" * 70)
426
427         print(f"Booking ID : {booking_id}")
428         print(f"Customer   : {booking['name']}")
429         print(f"Movie      : {booking['movie']}")
430         print(f>Show       : {booking['show']}")
431         print(f"Seats      : {' '.join(booking['seats'])}")
432         print(f"Tickets    : {booking['quantity']}")
433         print(f"Amount     : Rs. {booking['total']:.2f}")
434
435     print("-" * 70)
436
437 # =====
438 #             SEAT AVAILABILITY
439 # =====
440 # =====
441
442 def check_seat_availability():
443
444     print("=====SEAT AVAILABILITY=====")
445
446     movie_id = get_movie_choice()
447     show_id = get_show_choice()
448
449     initialize_seats(movie_id, show_id)
450

```

```

451     key = create_show_key(movie_id, show_id)
452
453     available = len(booked_seats[key])
454     total_seats = len(rows) * seats_per_row
455     booked = total_seats - available
456
457     print("Movie:", movies[movie_id][0])
458     print("Show :", show_times[show_id])
459
460     print(f"Total Seats      : {total_seats}")
461     print(f"Booked Seats     : {booked}")
462     print(f"Available Seats  : {available}")
463
464     display_seats(movie_id, show_id)
465
466 # =====
467 #                               MAIN MENU
468 # =====
469
470 def main_menu():
471     while True:
472
473         print("=====MAIN MENU=====")
474
475         print("1. Display Movies")
476         print("2. Display Show Timings")
477         print("3. Check Seat Availability")
478         print("4. Book Movie Ticket")
479         print("5. View Booking")
480         print("6. Cancel Booking")
481         print("7. Display All Bookings")
482         print("8. Exit")
483
484         print("-" * 70)
485
486         choice = input(
487             "Enter your choice (1-8): "
488         ).strip()
489
490         if choice == "1":
491             display_movies()

```

```

496         elif choice == "2":
497             display_show_times()
498
499         elif choice == "3":
500             check_seat_availability()
501
502         elif choice == "4":
503             book_ticket()
504
505         elif choice == "5":
506             view_booking()
507
508         elif choice == "6":
509             cancel_booking()
510
511         elif choice == "7":
512             display_all_bookings()
513
514         elif choice == "8":
515             print("THANK YOU FOR USING MOVIE TICKET BOOKING SYSTEM")
516
517             break
518
519     else:
520         print("Invalid choice!")
521         print("Please select a number between 1 and 8.")
522
523 # =====
524 #                               PROGRAM START
525 # =====
526
527 if __name__ == "__main__":
528     main_menu()

```